

**LAPORAN TUGAS BESAR  
IF25-21013 STRATEGI ALGORITMA**

**Syntax Terror**



**Oleh:**

Bagas Hari Muthi - 124140128

Arief Fandi Satria - 124140020

Arya Dimar Fath - 124140212

**PROGRAM STUDI TEKNIK INFORMATIKA  
INSTITUT TEKNOLOGI SUMATERA  
2026**

## Daftar Isi

|                                               |           |
|-----------------------------------------------|-----------|
| <b>BAB I Deskripsi Tugas.....</b>             | <b>2</b>  |
| 1.1 Spesifikasi Wajib.....                    | 2         |
| 1.2 Spesifikasi Bonus.....                    | 2         |
| <b>BAB II Landasan Teori.....</b>             | <b>4</b>  |
| 2.1 Dasar Teori.....                          | 4         |
| 2.2 Cara Kerja Program.....                   | 4         |
| 2.2.1 Cara Bot Beraksi.....                   | 4         |
| 2.2.2 Implementasi Algoritma Greedy.....      | 4         |
| 2.2.3 Cara Menjalankan Bot.....               | 5         |
| <b>BAB III Aplikasi Strategi Greedy.....</b>  | <b>5</b>  |
| 3.1 Proses Mapping Persoalan.....             | 5         |
| 3.2 Eksplorasi Solusi Greedy.....             | 6         |
| 3.3 Analisis Efisiensi dan Efektivitas.....   | 7         |
| 3.4 Strategi yang Dipilih.....                | 7         |
| <b>BAB IV Implementasi dan Pengujian.....</b> | <b>7</b>  |
| 4.1 Pseudocode.....                           | 7         |
| 4.1.1 Overclock (Main Bot).....               | 7         |
| 4.1.2 Deadlock (Alt Bot 1).....               | 8         |
| 4.1.3 Ghost (Alt Bot 2).....                  | 8         |
| 4.1.4 Rhino (Alt Bot 3).....                  | 9         |
| 4.2 Struktur pada Bot Utama.....              | 9         |
| 4.2.1 Struktur Data.....                      | 9         |
| 4.2.2 Fungsi dan Prosedur.....                | 9         |
| 4.3 Pengujian.....                            | 10        |
| 4.3.1 Pengujian 1.....                        | 10        |
| 4.3.2 Pengujian 2.....                        | 10        |
| 4.3.3 Pengujian 3.....                        | 10        |
| 4.3.4 Kejadian Unik.....                      | 10        |
| 4.4 Analisis Pengujian.....                   | 11        |
| <b>BAB V Kesimpulan dan Saran.....</b>        | <b>11</b> |
| 5.1 Kesimpulan.....                           | 11        |
| 5.2 Saran.....                                | 11        |
| <b>Lampiran.....</b>                          | <b>12</b> |
| <b>Daftar Pustaka.....</b>                    | <b>12</b> |

# BAB I Deskripsi Tugas

## 1.1 Spesifikasi Wajib

- Buatlah 4 bot (1 utama dan 3 alternatif, **tetap dikumpulkan 1 bot yang terbaik saja**) dalam bahasa C# ([.net](#)) yang mengimplementasikan *algoritma Greedy* pada *bot* permainan Robocode Tank Royale dengan tujuan memenangkan permainan.
- Tugas dikerjakan berkelompok dengan anggota **minimal 2 orang** dan **maksimal 3 orang**.
- Strategi *greedy* yang diimplementasikan setiap kelompok harus dikaitkan dengan fungsi objektif dari permainan ini, yaitu memperoleh skor setinggi mungkin pada akhir pertempuran. Hal ini dapat dilakukan dengan mengoptimalkan komponen skor yang telah dijelaskan diatas.
- Strategi *greedy* yang diimplementasikan **harus berbeda** untuk setiap bot yang diimplementasikan dan setiap strategi *greedy* harus menggunakan **heuristic** yang berbeda.
- **Bot yang dibuat TIDAK BOLEH sama dengan SAMPEL yang diberikan sebagai CONTOH. Baik dari starter pack maupun dari repository engine asli.**
- Buatlah strategi *greedy* terbaik, karena setiap **bot utama** dari masing-masing kelompok akan diadu dalam kompetisi Tubes.
- Strategi *greedy* yang kelompok anda buat harus **dijelaskan dan ditulis secara eksplisit** pada laporan, karena akan diperiksa saat demo apakah strategi yang dituliskan sesuai dengan yang diimplementasikan.
- Setiap kelompok dapat menggunakan kreativitas yang bermacam macam dalam menyusun strategi *greedy* untuk memenangkan permainan. Implementasi pemain **harus dapat dijalankan pada game engine** yang telah disebutkan diatas serta dapat dikompetisikan dengan bot dari kelompok lain.
- Program harus mengandung komentar yang jelas, dan untuk setiap strategi *greedy* yang disebutkan, harus dilengkapi dengan **kode sumber yang dibuat**. Artinya semua strategi harus diimplementasikan
- Mahasiswa disarankan membaca [dokumentasi](#) dari *game engine*. Perlu diperhatikan bahwa game engine yang digunakan untuk tubes ini **SUDAH DIMODIFIKASI**. Untuk Perubahan dapat dilihat pada bagian starter pack.

## 1.2 Spesifikasi Bonus

- (maks 10) Membuat video tentang aplikasi *greedy* pada bot serta simulasinya pada game kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan menampilkan wajah dari setiap anggota kelompok. Untuk contoh video tubes stima tahun-tahun sebelumnya dapat dilihat di Youtube dengan kata kunci “Tubes Stima”, “strategi algoritma”, “Tugas besar stima”, dll. **Semakin menarik video, maka semakin banyak poin yang diberikan.**
- (maks 10) Beberapa kelompok pemenang lomba kompetisi akan mendapatkan nilai tambahan berdasarkan posisi yang diraih.

- Jika terdapat kesulitan selama mengerjakan tugas besar sehingga memerlukan bimbingan, maka dapat melakukan asistensi tugas besar kepada asisten (opsional). Dengan catatan asistensi hanya bersifat membimbing, bukan memberikan “jawaban”.
- Bot yang telah dibuat akan dikompetisikan dengan kelompok lain dan disaksikan oleh seluruh peserta kuliah. Terdapat hadiah menarik bagi kelompok yang memenangkan kompetisi.
- Setiap kelompok harap mengisi nama kelompok dan anggotanya pada link berikut, paling lambat **Minggu, 26 April 2026**.
  - RA : <https://forms.gle/vudRWyHFXzWZaiKA7>
  - RB : <https://s.itera.id/TubesStima-RB26>
  - RC : <https://s.itera.id/TubesStigma-RC26>
  - RD : <https://forms.gle/z9Wa7rmfZJfxwQhY8>
- Program disimpan dalam repository yang bernama **Tubes\_NamaKelompok** dengan nama kelompok sesuai dengan yang di sheets diatas. Berikut merupakan struktur dari isi repository tersebut:



- Folder src berisi semua source code
- Folder doc berisi laporan tugas besar dengan format **NamaKelompok.pdf** dengan ketentuan isi pada **isi laporan**.
- README untuk tata cara penggunaan yang minimal berisi:
  - Penjelasan singkat algoritma greedy yang diimplementasikan untuk setiap bot yang dibuat
  - Requirement program dan instalasi tertentu bila ada
  - Command atau langkah-langkah dalam meng-compile atau build program
  - Author (identitas pembuat)
- Sangat disarankan untuk menggunakan [semantic commit](#). Buatlah release dengan format **v1.x** dengan x adalah nomor revisi dimulai dari revisi 0. Contoh v1.0 untuk release pertama, v1.1 untuk revisi selanjutnya.
- Pastikan untuk membuat repository bersifat **Public** paling lambat **H+1 deadline (1 hari setelah deadline)**. Sebelum deadline repository harus bersifat **Private**.

## **BAB II Landasan Teori**

### **2.1 Dasar Teori**

Algoritma greedy adalah sebuah pendekatan penyelesaian masalah yang membentuk solusi langkah demi langkah. Pada setiap langkah, algoritma ini akan mengambil keputusan yang paling optimal secara lokal pada saat itu juga, tanpa memikirkan konsekuensi di masa depan. Harapan utamanya adalah bahwa kumpulan dari setiap keputusan lokal yang optimal ini akan mengarah pada solusi optimal secara keseluruhan.

Dalam persoalan optimasi, algoritma greedy secara formal dibangun oleh enam elemen dasar, yaitu:

1. Himpunan Kandidat (C): Kumpulan elemen atau aksi yang dapat dipilih pada setiap langkah.
2. Himpunan Solusi (S): Himpunan yang menampung kandidat-kandidat yang telah dipilih untuk membentuk solusi akhir.
3. Fungsi Seleksi: Fungsi heuristik yang memilih kandidat paling menguntungkan dari himpunan kandidat untuk dimasukkan ke dalam himpunan solusi.
4. Fungsi Kelayakan: Fungsi yang memeriksa apakah suatu kandidat yang dipilih memenuhi aturan/kendala masalah sehingga layak dimasukkan ke dalam himpunan solusi.
5. Fungsi Solusi: Fungsi yang memeriksa apakah himpunan solusi yang terbentuk sudah lengkap dan memecahkan permasalahan.
6. Fungsi Objektif: Fungsi yang mengukur nilai pencapaian dari solusi, bisa berupa memaksimalkan atau meminimumkan suatu nilai.

### **2.2 Cara Kerja Program**

#### **2.2.1 Cara Bot Beraksi**

Permainan Robocode Tank Royale berjalan dalam unit waktu terkecil yang disebut turn (giliran). Pada setiap turn-nya, program mengirimkan perintah ke server melalui API. Perintah ini meliputi rotasi (menggerakkan radar, meriam, atau badan tank), pergerakan maju dan mundur, serta penembakan

Bot menerima informasi lingkungan sekitarnya melalui arsitektur Event-Driven dari engine permainan. Sebagai contoh, ketika radar bot mendeteksi keberadaan bot musuh, server akan memicu ScannedBotEvent. Bot bereaksi terhadap event tersebut, lalu melakukan kalkulasi geometri, seperti mengukur jarak absolut DistanceTo dan sudut hadap DirectionTo, dan kemudian memerintahkan meriam untuk membidik dan badan tank untuk bermanuver pada turn yang sama.

#### **2.2.2 Implementasi Algoritma Greedy**

Implementasi algoritma greedy pada bot dilakukan dengan memprogram bot agar mengambil tindakan yang paling "serakah" di setiap turn demi meraup poin skor setinggi mungkin. Berikut adalah cara kerjanya berdasarkan kode bot utama kami:

1. Seleksi Kandidat: Saat fungsi OnScannedBot berjalan, bot secara serakah akan terus melacak target dengan sisa jarak terdekat ( $\text{distanceToEnemy} < \text{minDistance}$ )

agar meminimalisir peluru meleset akibat proyektil yang terbang terlalu lama di udara.

2. Fungsi Kelayakan: Sebelum meriam menembak, bot memverifikasi kelayakannya yaitu memastikan suhu meriam sudah dingin ( $\text{GunHeat} == 0$ ) dan energi bot cukup ( $\text{Energy} > 1.0$ ). Untuk pergerakan, bot memiliki fungsi `CheckWallCollision()` untuk mencegah bot menabrak batas arena dan terkena wall damage.
3. Fungsi Objektif: Mengingat aturan tugas besar menggunakan Turn Limit (pertandingan bisa berhenti secara tiba-tiba), bot tidak bisa hanya diam. Bot ini memaksimalkan bullet damage dengan menggunakan daya proyektil maksimal secara dinamis ( $\text{Fire}(3.0)$  dari jarak dekat, dan  $\text{Fire}(1.0)$  dari jarak jauh). Secara serakah, jika musuh yang terdeteksi sedang sekarat ( $\text{e.Energy} < 15$ ) di jarak dekat, bot akan berubah prioritas untuk menabrak musuh tersebut (aksi Ramming) untuk mengklaim Ram Damage ganda dan poin bonus kelipatan 30%.

### 2.2.3 Cara Menjalankan Bot

Proses mengkompilasi dan menguji bot pada game engine asisten dilakukan melalui tahapan berikut:

1. Buka aplikasi GUI Robocode Tank Royale dengan mengeksekusi file .jar (java -jar robocode-tankroyale-gui-0.30.0.jar).
2. Mulai Local Server melalui menu tab Server > Start Local Server.
3. Lakukan kompilasi kode source C# dengan menjalankan perintah dotnet build dan dotnet run melalui terminal. Proses ini akan memulai bot dan menghubungkannya dengan local server
4. Untuk memulai pertempuran, buka tab Battle > Start Battle. Pilih bot-bot yang ingin diadu, kemudian Add, dan klik Start Battle untuk melihat simulasi bot berjalan di arena.

## BAB III Aplikasi Strategi Greedy

### 3.1 Proses Mapping Persoalan

Sebelum merumuskan alternatif solusi, persoalan pertempuran tank dalam Robocode Tank Royale perlu dipetakan terlebih dahulu ke dalam enam elemen dasar algoritma greedy:

1. Himpunan Kandidat (C): Semua kemungkinan aksi yang dapat dilakukan bot pada setiap turn, seperti bergerak maju/mundur dengan jarak tertentu, berbelok dengan sudut tertentu, memutar meriam/radar, dan menembak dengan berbagai tingkat kekuatan.
2. Himpunan Solusi (S): Kumpulan keputusan aksi yang telah dieksekusi oleh bot dari awal pertandingan hingga turn saat ini.

3. Fungsi Seleksi: Heuristik yang digunakan bot untuk memilih aksi atau target paling menguntungkan dari himpunan kandidat pada saat itu (misalnya memilih musuh terdekat atau aksi yang paling aman).
4. Fungsi Kelayakan: Kondisi yang memvalidasi apakah suatu aksi legal untuk dieksekusi.
5. Fungsi Solusi: Kondisi yang mengevaluasi apakah pertempuran telah berakhir (bot hancur, bot menjadi pemenang terakhir, atau waktu turn habis).
6. Fungsi Objektif: Memaksimalkan total perolehan skor akhir yang diakumulasikan dari Bullet Damage, Kill Bonus, Survival, dan Ramming Bonus.

### 3.2 Eksplorasi Solusi Greedy

Untuk menemukan optimasi terbaik, dikembangkan 1 bot utama dan 3 alternatif bot yang masing-masing mengimplementasikan algoritma greedy dengan kriteria "Fungsi Seleksi" yang berbeda dalam mendapatkan poin. Berikut adalah pemaparan keempat alternatif tersebut:

1. Alternatif 1: Bot Overclock (Nearest Target Greedy)
  - Heuristic: Selalu memprioritaskan target dengan jarak terdekat (minDistance).
  - Cara Kerja: Bot secara serakah membidik musuh yang paling dekat untuk meminimalisir waktu terbang peluru, sehingga probabilitas peluru mengenai sasaran menjadi maksimal.
2. Alternatif 2: Bot Deadlock (Firepower Greedy)
  - Heuristic: Mengunci satu target secara terus-menerus dan mengeksekusi tembakan dengan firepower maksimal tanpa menghiraukan musuh lain.
  - Cara Kerja: Bot secara serakah mencari musuh yang hampir mati dan menembakkan peluru dengan daya maksimal untuk segera mendapatkan poin Kill Bonus sebesar 20%, mengabaikan efisiensi energinya sendiri.
3. Alternatif 3: Bot Ghost (Survival Greedy)
  - Heuristic: Memilih pergerakan yang memposisikan bot pada jarak terjauh dari semua musuh yang terdeteksi.
  - Cara Kerja: Bot serakah terhadap poin Survival dan Last Survivor Bonus. Bot menghindari pertempuran terbuka, selalu kabur ke area kosong, dan hanya menembak jika dipaksa.
4. Alternatif 4: Bot Rhino (Ramming Bonus Greedy)
  - Heuristic: Memprioritaskan pergerakan dengan jarak tempuh terpendek menuju koordinat musuh untuk melakukan kontak fisik.
  - Cara Kerja: Bot secara serakah mengejar poin Ramming Bonus dengan terus mendesak maju dan menabrak target tanpa memedulikan tembakan musuh.

### 3.3 Analisis Efisiensi dan Efektivitas

Keempat implementasi heuristik di atas dievaluasi untuk melihat mana yang secara konsisten menghasilkan skor total tertinggi di akhir pertandingan. Berikut adalah analisisnya

| No. | Nama Bot  | Kriteria Seleksi | Keunggulan                                                         | Kelemahan                                                            |
|-----|-----------|------------------|--------------------------------------------------------------------|----------------------------------------------------------------------|
| 1   | Overclock | Target terdekat  | Seimbang antara akurasi damage dan efisiensi peluru.               | Sering berpindah target jika ada musuh lain yang tiba-tiba mendekat. |
| 2   | Deadlock  | Kekuatan tembak  | Sangat kuat dalam duel 1 lawan 1,                                  | Cepat kehabisan energi.                                              |
| 3   | Ghost     | Jarak aman       | Sulit mati di awal, sering mendapat survival bonus.                | Kalah dalam pengumpulan poin damage total.                           |
| 4   | Rhino     | Bonus tabrakan   | Mendapat poin burst besar dari tabrakan dan merusak formasi musuh. | Menerima damage balik akibat tabrakan.                               |

### 3.4 Strategi yang Dipilih

Bot Overclock dipilih sebagai solusi utama karena paling seimbang dalam menghadapi aturan turn limit. Dengan memprioritaskan serangan pada target terdekat, bot ini secara konsisten memaksimalkan perolehan bullet damage dengan tingkat akurasi tinggi, serta lebih stabil dibandingkan alternatif lain yang terlalu boros energi, terlalu pasif, atau terlalu berisiko terkena self-damage akibat tabrakan.

## BAB IV Implementasi dan Pengujian

### 4.1 Pseudocode

#### 4.1.1 Overclock (Main Bot)

```
FUNCTION OnScannedBot(ScannedBotEvent e):  
    distanceToEnemy = DistanceTo(e.X, e.Y)  
  
    // Fungsi Seleksi: Mengunci target terdekat  
    IF targetName == NULL OR distanceToEnemy < minDistance OR e.ScannedBotId  
    == targetName:  
        targetName = e.ScannedBotId  
        minDistance = distanceToEnemy  
  
    targetDir = DirectionTo(e.X, e.Y)  
    gunTurn = CalcDeltaAngle(GunDirection, targetDir)  
    TurnGunRight(gunTurn)
```



```

// Fungsi Kelayakan: Mengecek kesiapan senjata dan energi
IF GunHeat == 0 AND Energy > 1.0:
    IF distanceToEnemy > 300: firePower = 1.0
    ELSE IF distanceToEnemy > 150: firePower = 2.0
    ELSE: firePower = 3.0
    Fire(firePower)

// Fungsi Seleksi Pergerakan (Tabrak / Kejar / Menghindar)
bodyTurn = CalcBearing(targetDir)
IF distanceToEnemy < 120 AND e.Energy < 15:
    TurnRight(bodyTurn)
    Forward(60 * moveDirection) // Menabrak musuh
ELSE IF distanceToEnemy > 150:
    TurnRight(bodyTurn)
    Forward(40 * moveDirection) // Mengejar musuh
ELSE:
    TurnRight(bodyTurn + 90) // Bergeser menyamping
    CheckWallCollision()
    Forward(40 * moveDirection)

```

#### 4.1.2 Deadlock (Alt Bot 1)

```

FUNCTION OnScannedBot(ScannedBotEvent e):
    // Fungsi Seleksi: Mengunci target pertama secara permanen
    IF targetName == NULL:
        targetName = e.ScannedBotId

    IF e.ScannedBotId == targetName:
        targetDir = DirectionTo(e.X, e.Y)
        gunTurn = CalcDeltaAngle(GunDirection, targetDir)
        TurnGunRight(gunTurn)

    // Fungsi Kelayakan: Selalu gunakan daya tembak maksimal
    IF GunHeat == 0:
        Fire(3.0)

```

#### 4.1.3 Ghost (Alt Bot 2)

```

FUNCTION OnScannedBot(ScannedBotEvent e):
    distanceToEnemy = DistanceTo(e.X, e.Y)

    // Fungsi Seleksi: Menjauh jika musuh berada di zona bahaya
    IF distanceToEnemy < 350:
        escapeAngle = e.Bearing + 90
        TurnRight(escapeAngle)

```

```
Forward(100 * moveDirection) // Bergerak menjauh
```

```
// Fungsi Kelayakan: Tembakan defensif hemat energi
```

```
IF GunHeat == 0:
```

```
    Fire(1.0)
```

#### 4.1.4 Rhino (Alt Bot 3)

```
FUNCTION OnScannedBot(ScannedBotEvent e):
```

```
    targetName = e.ScannedBotId
```

```
    distanceToEnemy = DistanceTo(e.X, e.Y)
```

```
// Fungsi Seleksi: Rute langsung menuju koordinat musuh
```

```
    targetDir = DirectionTo(e.X, e.Y)
```

```
    bodyTurn = CalcBearing(targetDir)
```

```
    TurnRight(bodyTurn)
```

```
// Fungsi Kelayakan: Cek batas tembok sebelum menabrak
```

```
    CheckWallCollision()
```

```
    Forward(distanceToEnemy) // Eksekusi tabrakan
```

```
// Tembak saat terjadi kontak fisik jarak dekat
```

```
IF GunHeat == 0 AND distanceToEnemy < 50:
```

```
    Fire(3.0)
```

## 4.2 Struktur pada Bot Utama

### 4.2.1 Struktur Data

- string targetName: Digunakan untuk menyimpan identitas musuh (ScannedBotId).
- double minDistance: Digunakan untuk menyimpan nilai jarak terdekat saat ini.
- int moveDirection: Struktur data state untuk menentukan arah pergerakan (maju atau mundur).
- ScannedBotEvent e: Objek event yang disediakan API untuk menangkap data musuh secara instan (posisi X, Y, energi, dan ID). Objek ini berfungsi sebagai struktur data utama yang diproses oleh algoritma.

### 4.2.2 Fungsi dan Prosedur

- OnScannedBot(ScannedBotEvent e): Prosedur utama yang mengeksekusi logika greedy setiap kali ada musuh terdeteksi. Di sini, bot melakukan perbandingan jarak ( $\text{distanceToEnemy} < \text{minDistance}$ ) untuk menentukan apakah musuh saat ini layak menjadi target utama berdasarkan kriteria Greedy by Profit.
- Fire(double firePower) (Fungsi Kelayakan): Fungsi ini dipanggil hanya jika kondisi  $\text{GunHeat} == 0$  dan  $\text{Energy} > 1.0$  terpenuhi. Prosedur ini menerapkan heuristik greedy

dengan memberikan daya tembak lebih besar (3.0) pada jarak dekat untuk memaksimalkan bullet damage secara instan.

- CheckWallCollision() (Fungsi Kelayakan): Prosedur yang memeriksa apakah koordinat bot (X atau Y) mendekati batas arena (50 unit dari tembok). Jika true, maka moveDirection dibalik, berfungsi sebagai feasibility checker agar bot tidak terkena penalti wall damage yang merugikan fungsi objektif skor.
- TurnGunRight(double gunTurn) (Fungsi Seleksi): Mengarahkan meriam ke target berdasarkan sudut terpendek. Prosedur ini memastikan bot selalu dalam kondisi siap menembak target yang telah diseleksi sebagai yang paling optimal secara lokal.

## 4.3 Pengujian

### 4.3.1 Pengujian 1

| Results for 10 rounds |                      |             |          |             |             |              |          |           |      |      |      |
|-----------------------|----------------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name                 | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | Overclock (Main) 1.0 | 1090        | 250      | 50          | 670         | 112          | 7        | 0         | 5    | 2    | 0    |
| 2                     | Deadlock (Alt 1) 1.0 | 679         | 50       | 10          | 585         | 26           | 7        | 0         | 2    | 5    | 0    |

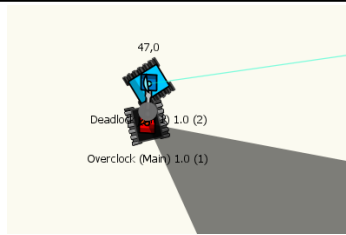
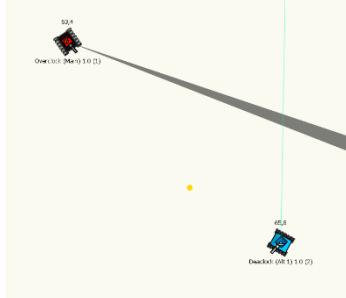
### 4.3.2 Pengujian 2

| Results for 10 rounds |                      |             |          |             |             |              |          |           |      |      |      |
|-----------------------|----------------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name                 | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | Overclock (Main) 1.0 | 1143        | 250      | 50          | 752         | 88           | 2        | 0         | 6    | 1    | 0    |
| 2                     | Deadlock (Alt 1) 1.0 | 659         | 50       | 10          | 575         | 20           | 4        | 0         | 1    | 6    | 0    |

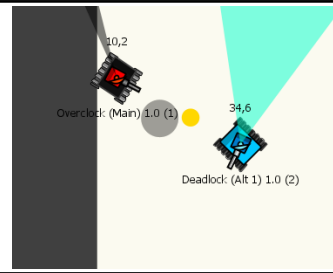
### 4.3.3 Pengujian 3

| Results for 10 rounds |                      |             |          |             |             |              |          |           |      |      |      |
|-----------------------|----------------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name                 | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | Overclock (Main) 1.0 | 1006        | 200      | 40          | 676         | 89           | 0        | 0         | 5    | 2    | 0    |
| 2                     | Deadlock (Alt 1) 1.0 | 703         | 100      | 20          | 542         | 41           | 0        | 0         | 2    | 5    | 0    |

### 4.3.4 Kejadian Unik

| Kejadian                                                      | Foto                                                                                 |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Overclock melakukan ramming.                                  |  |
| Overclock menembak dari jarak jauh menggunakan firePower = 1. |  |

Overclock menembak dari jarak dekat menggunakan firePower = 3.



#### 4.4 Analisis Pengujian

Strategi greedy terbukti berhasil mencapai nilai optimal pada bot Overclock. Dengan memprioritaskan musuh terdekat, Overclock mampu menyeimbangkan damage output dan efisiensi peluru secara konsisten di setiap turn. Sebaliknya, Deadlock gagal mencapai skor optimal secara konsisten. Meskipun secara teoretis membidik satu target dengan daya penuh adalah langkah agresif, pada praktiknya strategi ini justru sering terjebak dalam local optimum yang merugikan.

Strategi greedy yang kaku seperti Deadlock sangat lemah jika musuh yang dikunci menjauh dan musuh lain mendekat. Deadlock tidak memiliki fleksibilitas untuk beralih target, sehingga efisiensi skornya anjlok karena ia kehilangan waktu membidik target yang sulit dicapai sementara dirinya sendiri tidak memberikan damage pada musuh yang berada di depan matanya.

## BAB V Kesimpulan dan Saran

### 5.1 Kesimpulan

Implementasi algoritma greedy pada Robocode Tank Royale terbukti efektif sebagai mekanisme pengambilan keputusan berbasis status lingkungan secara real-time. Berdasarkan eksperimen terhadap empat alternatif heuristik, disimpulkan bahwa strategi yang diimplementasikan pada bot Overclock merupakan metode yang paling optimal dalam memaksimalkan total skor di bawah batasan turn limit. Strategi ini mampu menyeimbangkan akurasi tembakan dan efisiensi energi secara adaptif dibandingkan strategi statis lainnya, seperti Deadlock yang cenderung kaku dalam situasi arena yang dinamis atau Ghost yang terlalu pasif dalam perolehan skor kerusakan.

### 5.2 Saran

Untuk pengembangan selanjutnya, disarankan agar implementasi algoritma greedy dibuat lebih adaptif dengan mengintegrasikan state machine yang memungkinkan bot berpindah heuristik secara dinamis sesuai kondisi energi, menambahkan perhitungan untuk memprediksi posisi musuh di masa depan, serta menambahkan algoritma penghindaran peluru berbasis vektor untuk meminimalkan damage musuh tanpa harus mengorbankan agresifitas bot dalam perolehan skor.

## Lampiran

**Repository Github:** [https://github.com/braxys/Tubes\\_SyntaxTerror](https://github.com/braxys/Tubes_SyntaxTerror)

**Video:** <https://youtu.be/fPsS68l01ZA?si=xlPegd9DnENb0QMQ>

| No. | Poin                                                              | Ya | Tidak |
|-----|-------------------------------------------------------------------|----|-------|
| 1   | Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten. | ✓  |       |
| 2   | Membuat 4 solusi greedy dengan heuristic yang berbeda.            | ✓  |       |
| 3   | Membuat laporan sesuai dengan spesifikasi.                        | ✓  |       |
| 4   | Membuat video bonus dan diunggah pada Youtube.                    | ✓  |       |

## Daftar Pustaka

ITERA. (2026). *Robocode Starter Guide*. Institut Teknologi Sumatera.

Microsoft. (2026). *C# Documentation: Get Started with C#*. Diakses dari <https://learn.microsoft.com/en-us/dotnet/csharp/>

Robocode Tank Royale. (2026). *Robocode Tank Royale Documentation*. Diakses dari <https://robocode-dev.github.io/tank-royale/>